

سوال ۱ کد ۸: (وقت نشد با بقیه سوال ۱ بفرستم)

تابع این کد دو عدد می گیرد، اولی را با دو برابر دومی و دومی را با دو برابر اولی (بدون تغییر) جمع می کند. بعد دو حاصل را با هم جمع می کند. یعنی نهایتاً سه برابر مجموع دو عدد را می دهد.

با این حساب خروجی پرینت اولی واضح است. $(3 * (4 + 6) = 20, 4, 6)$

دومی همان تابع است که ورودیش باید از نوع `int*` باشد یعنی آدرس دو عدد را بدهیم سه برابر مجموعشان را بدهد.

پس باید به `f2`، `&a` و `&b` را بدهیم تا همان خروجی `f1` را به ما بدهد.

سوال ۲:

کد ۱:

حالت اول:

`a*` از نوع اینت است پس سایش ۴ می شود. یعنی پرینت اول ۴ می دهد.

پرینت دوم ۸ است چون `a` پوینتری از نوع اینت استار است و سایش اینت استار ۸ است.

پرینت سوم آدرس خانه اول فضای `calloc` شده را می دهد که یک عدد تصادفی در مبنای ۱۶ است.

پرینت چهارم: وقتی یکی به `a` اضافه می شود چون `a` از نوع اینت است و سایش ۴ است پوینتر به ۴ تا خانه بعد از خانه پرینت سوم می رود یعنی عدد پرینت چهارم ۴ تا از پرینت سوم بزرگتر است.

پرینت پنجم: خانه هایی که `calloc` میشوند در ابتدا مقدار همه شان صفر است مگر اینکه مقداردهی کنیم که نکردیم. `*a` مقدار خانه اول این حافظه است که صفر است پس صفر چاپ می کند.

حالت دوم:

دقیقا مثل حالت اول است. فقط خروجی `calloc` در حالت اول `void*` بود اینجا `int*` است اما چون در هر دو `a` را از نوع `int*` تعریف کردیم جواب ها فرقی نمی کند.

حالت سوم:

این هم مثل دو حالت قبل است. `Calloc` انگار آرایه ای دو بعدی است که در دو حالت اول سائز هر سطر آن به اندازه `sizeof(char)` یعنی ۱ بوده حالا سائز هر سطر آن ۴ شده. ولی این روی خروجی تاثیری ندارد.

حالت چهارم:

دو پرینت اول عوض می شود چون `a` از نوع `char*` تعریف شده. سائز `char` ۱ است پس پرینت اول ۱ می شود. سائز `char*` وکلا هر `type` یک ستاره ۸ است پس خروجی دوم ۸ می شود.

• خروجی سوم و چهارم در هر مرحله عوض می شود چون با هر بار اجرای برنامه یک فضای جدید `calloc` می شود که لزوما آدرسش با آدرس فضای قبلی یکی نیست.

حالت پنجم:

این هم مثل حالت ۴ است. تعداد و سائز خانه های `malloc` شده روی خروجی تاثیری ندارد و پرینت شده ها به `type` متغیر استار بستگی دارند.

حالت ۶:

این هم مثل حالت ۴ است چون گفتیم ورودی و خروجی malloc در type و در نتیجه در سائز متغیر a اثری ندارد و چون type و در نتیجه سائز a در هر دو حالت یکیست آنچه پرینت می شود هم همان است به جز آدرس یعنی پرینت f سوم و چهارم چون گفتیم آدرس مدام عوض می شود.

حالت ۷:

سائز float ۴ است اول ۴ چاپ می کند.
سائز float* ۸ است پس دومین پرینت، ۸ است.
بقیه مثل حالت اول.

حالت ۸:

باز سائز double ۸ است اول ۸ چاپ می شود.
سائز double* ۸ است بعد ۸ چاپ می شود.
بقیه مثل قبل

حالت ۹:

همه چیز مثل حالت اول است فقط فرق malloc با calloc این است که calloc مقدار اولیه صفر به همه خانه های گرفته شده می دهد اما malloc نمی دهد. پس پرینت پنجم که مقدار اولین خانه گرفته شده است لزوما صفر نیست.

کد ۲:

Runtime error می دهد برای اینکه یک اشاره گر را که به خانه اول یک اینت اشاره میکند، خانه اختصاص داده شده به آن را آزاد می کنیم. هر بلایی سر پوینتر بیاید سر متغیری که پوینتر به آن اشاره می کند هم می آید یعنی خانه b هم free می شود. بعد مثل این است:

```
Int main()  
{Int i;  
Printf(“%d”, i);  
Return ۰;  
}
```

یعنی می خواهد مقدار آدرسی را پرینت کند که وجود ندارد.

کد ۳:

Calloc ۵ تا خانه ۴ تایی (۴ سائز اینت است) را اختصاص می دهد و خروجی آن آدرس اولین این خانه هاست. مقدار اولیه این خانه ها صفر است. اما در پرینت ، a چاپ می شود نه *a پس مقدار مهم نیست. همان آدرس اولین بار در مبنای ۱۰ بار دوم در مبنای ۱۶ و بار سوم باز در مبنای ۱۰ به صورت چاپ می شود unsigned (که چون عدد مثبت است و از یک حدی هم بزرگتر نیست با مبنای ۱۰ آن فرقی نمیکند) حالا اگر printf(“%d”, *a) باشد صفر چاپ می کند.

کد ۴:

اولاً c در خط ۸ برنامه تعریف نشده. بعد اگر در جایی بالاتر $\text{int } c = ۲۱$ را تعریف کنیم فایده ای ندارد چون باید اول یک فضای `alloc` شده باشد که بعد آن را `realloc` کنیم. مثلاً خط ۸ را بنویسیم

```
char* b = realloc(a, sizeof(char))
```

درست می شود.

خروجی `malloc` اول یک پوینتر است که آدرس اولین خانه از $۵ * ۴$ خانه گرفته شده را می دهد.

در پرینت اول این آدرس چاپ می شود

در پرینت دوم یکی به این پوینتر اضافه می شود و چون پوینتر از نوع `int` است ۴ تا خانه می رود جلو. یعنی آدرس چاپ شده در این پرینت ۴ تا از آدرس درج شده در پرینت اول جلوتر است.

بعد وقتی `realloc` می کند این خانه ها را آزاد می کند و جای دیگری را در حافظه می گیرد.

وقتی `b` پرینت می شود یعنی آدرس خانه اول این حافظه جدید پرینت می شود.

وقتی `b+۱` پرینت می شود ، `b` چون از نوع `char` است و اندازه `char` ۱ است وقتی `b` یکی جلو می رود پوینتر ۱ واحد در حافظه جلو می رود و آدرسی که چاپ می شود فقط یکی از آدرس قبلی بیشتر است.